Daily Digest & Weekly Report — Product & System Design

1. Users and job-to-be-done

The Daily Digest is read by a growth lead or founder between 8 and 9 AM, on their phone, before standup. They are not doing analysis. They are doing triage. The question they're answering is "do I need to act on anything before my team syncs today?" — and they have 90 seconds to decide.

This means the digest has to be opinionated. Twelve metrics and a "form your own view" is a dashboard. The digest leads with the single most anomalous signal, flags whether the data is complete or suspect, and gives a directional interpretation with hedging proportional to the evidence. A founder doing INR 22.6L median daily revenue does not need to know impressions were up 3%. They need to know new-customer AOV on December 10 jumped 190.5% week-over-week and that the pattern is consistent with a sale event drawing a discounted-acquisition cohort — not organic demand.

The Weekly Report is a different job for a different reader mode. CMO, agency lead, or founder doing Sunday planning. The job is pattern recognition: which channels showed sustained movement, whether a single anomalous day was noise or the start of a trend, whether budget allocation should be revisited before next week begins. Daily answers "was yesterday unusual?" Weekly answers "what is the underlying story this week?" The reader tolerates more text and expects ranked tables alongside the narrative.

A campaign showing ROAS = 0.000 for the week of February 1 — search_google_002 spent budget and returned zero attributed orders — belongs in the Weekly Report with full context. It is a budget decision, not a morning alert.

2. What makes a metric movement "important"

Statistical scoring

```
final_score = (|z|/3 × 0.4 + |WoW|/100 × 0.4 + |MoM|/100 × 0.2) × business_weight
```

Three signals, weighted by their actionability. Z-score captures how far today's value is from its recent baseline relative to its own volatility. WoW catches sharp seven-day reversals. MoM catches slow drifts that WoW smooths over. Weights of 0.4 / 0.4 / 0.2 reflect the practical reality that D2C operational decisions are made on a 7-day horizon.

The z-score uses a 28-day rolling window with day-of-week adjustment — same-DoW mean and same-DoW standard deviation, so numerator and denominator come from the same distribution. This is non-negotiable on this dataset. Thursday is the strongest day of the week and Sunday is the weakest, with

a ~40% spread. Unadjusted z-scores would flag every Sunday as a downside anomaly and every Thursday as upside. The EDA confirmed four genuine |z| > 2 days across 160 days (Oct 8, Dec 10, Jan 1, Feb 7) — a reasonable false-positive rate.

Surfacing threshold is |z| > 2.0. |z| > 3.0 is flagged as a high-confidence anomaly.

Business weighting

Weights live in `ranker.py:BUSINESS_WEIGHTS` and multiply the statistical score:

Metric	Weight	Why
revenue	1.0	Direct P&L
orders	0.9	Volume proxy; leads revenue by hours
roas	0.8	Efficiency lever for budget decisions
aov	0.75	Cohort quality indicator
spend	0.7	Controllable input; less informative than outputs
new_customer_share	0.65	Acquisition health
attributed_revenue	0.6	Platform-reported; weakly correlated with Shopify
attributed_orders	0.55	Platform-reported volume
new_orders / returning_orders	0.5	Cohort split
cpc	0.4	Auction cost signal
cpm	0.35	Auction signal; contextual, not decisive
ctr	0.3	Creative signal; rarely actionable same-day
impressions / clicks	0.2	High noise, low decision value

A 50% impressions swing is usually auction dynamics, day-of-week, or pacing — and almost never needs a same-day decision. A 5% revenue dip is barely statistically anomalous but very actionable. The weights bake that asymmetry into the ranking, so a large ROAS movement on a low-spend campaign doesn't beat a smaller revenue movement at brand-scale.

The double-counting problem

Shopify emits three overlapping slices per metric per day. The daily total (`channel = ""`, `customer_type = ""`) is the brand's overall number. The channel slice splits revenue by referrer; ~91% lands in `unattributed`, so its value is numerically identical to the daily total on most days. The customer-type slice splits the same revenue by `new` vs `returning` and also sums to the daily total.

Scoring all rows naively triple-counts the same revenue and lets the largest single row (`unattributed`) dominate every digest. Worse, the daily total and the unattributed slice would surface as two findings with the same number.

`ranker.py:DEDUP_PAIRS` keeps the daily total as the headline and drops the `unattributed` channel slice. The remaining attributed slices (`direct`, `email`, `paid`, etc.) are still scored — they're

directional even if attribution is thin. Customer-type slices are scored separately because they answer a different question: cohort mix, not channel attribution.

The ratio metric artifact problem

Ratio metrics — ROAS, CTR, CPM, CPC — explode mathematically when the denominator approaches zero. `shopping_google_003` in the raw data shows WoW ROAS = +4,653% because its prior-week spend was INR 340. The number is not a 47x improvement. It's a near-zero denominator.

WoW and MoM deltas for ratio metrics are clipped at ±500% before scoring. The cap is high enough to preserve genuine large movements (doubling a ROAS from 1.0 to 2.0 is +100%, well under) and low enough to eliminate the artifact tail. Capped values are disclosed in the fact sentence with `[capped]` so the LLM communicates the caveat instead of treating it as a business insight.

---

3. Personalization

Cold start (under 14 days of data)

Fixed defaults. Business weights from the table above. DoW adjustment disabled (not enough same-weekday observations to be stable). Z-scores replaced with percentile rank within the available window. The output leads with "baseline statistics stabilise after 4 weeks of data" so the user calibrates expectations. Default profile is PROFILE_GROWTH on the assumption that a customer onboarding to an analytics tool is more likely to be acquisition-focused than retention-focused.

Warm customer (2–8 weeks)

At two weeks, DoW adjustment activates. At four weeks the 28-day rolling window fills and z-scores are statistically valid. The interesting work during this window is collecting profile signal — which findings get clicked, which trigger follow-up questions, which get scrolled past. None of that is implemented yet because no engagement data exists; the ranker is ready to accept per-customer weight adjustments when it does.

Mature customer (60+ days)

Three things become possible only at this horizon. Seasonality detection — Thursday spikes for this specific brand stop being flagged as anomalies. Campaign persistence classification — ephemeral test campaigns are filtered out of the ranker entirely. Engagement-based weight learning — per-customer business weights replace the global defaults. A mature digest is shorter than a cold-start digest, because the system has learned what to suppress.

The Jan 29 direct-channel anomaly — revenue +370.7% MoM, AOV INR 33,679 against baseline INR 12,312 — would lead a mature customer's digest because it's a cohort-level departure, not a seasonal artifact.

The two profiles implemented

Both profiles carry the same shape: `primary_metrics`, `known_concerns`, free-text `context_note`. Personalisation operates in two places.

Re-ranking at the online layer. `apply_profile_reranking()` multiplies `final_score` by 1.3 for any finding whose metric is in the customer's `primary_metrics`, then re-sorts. The same Dec 10 dataset produces a different leading finding for growth vs scale.

Prompt injection. The profile's priorities and known concerns are interpolated into the user message so the LLM's narrative emphasis matches the re-ranker's selection.

PROFILE_GROWTH — revenue, ROAS, new_customer_share, spend, attributed_revenue, new_orders. The question is "are we acquiring efficiently?"

PROFILE_SCALE — revenue, AOV, returning_orders, new_customer_share, orders. The question is "what's the quality of customers we're acquiring, and are we retaining them?"

The 1.3x multiplier is the cold-start prior — auditable, easy to explain, easy to replace by a learned per-customer weight once engagement data exists. It is not a learned model.

---

4. System architecture

Batch layer (nightly)

A scheduled job runs after Shopify and ad-platform exports settle. It ingests the three sources, applies dedup, computes rolling z-scores with DoW adjustment, calculates WoW and MoM deltas with ratio capping, applies business weights, and writes `ranked_findings.json`. Pure Python, deterministic, no LLM. Target runtime: under 4 minutes per tenant for 160 days of history.

A `data_quality.json` sidecar flags last-day completeness (is the export timestamp within 6 hours of midnight?), any channel with zero spend and non-zero attributed revenue (tracking error), and any day where Shopify revenue deviates more than 60% from the prior same-day-of-week (candidate incomplete export).

Online layer (per-report)

When a report is requested, the generator reads `ranked_findings.json`, applies `apply_profile_reranking()` (+30% on priority metrics), slices to top-5 daily / top-8 weekly, formats each finding as a fact sentence, and sends a Claude API call with `cache_control: ephemeral` on the shared system prompt. Target end-to-end latency: under 12 seconds at p95. Three retries with exponential backoff on transient failures. On API failure or numeric-grounding failure, `render_template_fallback()` emits a deterministic markdown rendering of the verified facts — ugly but always correct.

The deterministic / generative boundary

The boundary sits at `ranked_findings.json`. Everything upstream is deterministic and auditable. Every number in the final report was computed before the LLM was invoked. The LLM's job is exclusively to write hedged prose connecting pre-computed facts.

The reason for the boundary is unspectacular: LLMs are unreliable calculators and reliable writers. Asking an LLM to decide whether a ROAS movement is significant requires it to reason about baselines, volatility, and business context — tasks it gets wrong in unpredictable ways. Asking it to write

"search_google_002 recorded a ROAS of 0.000 for the week of February 1, meaning spend occurred with zero attributed conversions" is a task it performs consistently and well.

Data flow

```
Raw CSVs (meta_ads, google_ads, shopify)
        |
        ▼
[ Ingestion + dedup ]
   • drop shopify_unattributed (duplicates the total)
   • last-day completeness check
        |
        ▼
[ Scoring engine ]
   • 28-day rolling z (same-DoW mean + std)
   • WoW / MoM deltas (ratio metrics capped at 500%)
   • final_score = stat_score × business_weight
   • Sustained-signal boost (+10%/day for 3+ day streaks)
        |
        ▼
  ranked_findings.json  +  data_quality.json
        |
        ▼
[ Online report generator ]
   • apply_profile_reranking (+30% on priority metrics)
   • top-5 daily / top-8 weekly
   • STEADY block: 3 low-z high-weight metrics
   • Claude API (cached system prompt, retry, fallback)
   • validate_numeric_grounding (FACTS + STEADY + user + system)
        |
        ▼
   Daily Digest (.md, 5 sections) / Weekly Report (.md, 5 sections)
```

---

5. Top 3 failure modes and mitigations

Failure 1: Hallucinated causation

The LLM writes "revenue dropped because Meta CPM rose." The number is correct, the framing is fabricated. The brand owner cuts Meta. Revenue keeps dropping because the real cause was a broken checkout. Trust gone.

The cause is structural — LLMs are trained on millions of analytical reports and the "X dropped because Y rose" pattern is overwhelmingly common in training data. Without instruction the model defaults to confident causation.

Mitigation is three layers. The system prompt explicitly bans "because", "caused by", "due to" and lists allowed hedge phrases. The prompt cites the actual data constraint — r=0.12 between spend and revenue — so the model understands why the rule exists. After generation, `validate_numeric_grounding()` extracts every numeric token from the output, normalises it, and verifies it appears in the FACTS, STEADY, user-prompt, or system-prompt context. Ungrounded

numbers reject the output and trigger the template fallback. The check is logged with the specific tokens, so when it fires the failure is debuggable.

Failure 2: Alert fatigue

Naive |z| > 2 surfacing on this dataset would flag 5–10 metrics every day. A brand with 15 campaigns receives 40+ findings per morning. Users stop reading within a week. The product becomes noise.

Mitigation is five compounding mechanisms. DoW adjustment prevents Sunday-naturally-low from flagging every Sunday — most of the false-positive volume disappears here. Business weights ensure impressions z=2.5 can't beat revenue z=2.0. The top-N cap (5 daily, 8 weekly) is a hard limit regardless of how many signals exceed threshold. Sustained-signal boost in the weekly ranker promotes 3-day trends ahead of one-off spikes, so the weekly report tells the underlying story rather than the loudest one. The mandatory "what's holding steady" section, grounded in a separate STEADY FACTS block, gives the reader explicit reassurance that not everything is on fire.

Failure 3: Incomplete last-day data

The dataset's last date (Feb 7) shows Shopify revenue down 92%. That looks catastrophic. It is almost certainly because the export was generated mid-day and the day wasn't complete.

If the system leads with "revenue down 92%", the brand owner panics, opens an incident, escalates. Six hours later somebody figures out the export was mid-day. Trust gone for a different reason.

Mitigation is three layers. `is_data_quality_flag = True` is set on any finding whose target date equals the dataset's last date. The sort key pushes flagged findings to the bottom — they don't disappear (the user might still want to know) but they don't lead. The fact sentence is annotated with `[NOTE: possible incomplete data — last day of dataset]` so the LLM communicates the caveat instead of presenting it as a confirmed movement. In production this would be replaced by a proper freshness check on the export timestamp and expected row count; the last-day heuristic is a reasonable proxy for the prototype.

---

6. What would I measure to know this is working

Product metrics, not model metrics. The model isn't the product — the morning habit is.

Open rate. Does the customer open the digest every morning, or only when something is wrong? An open rate that decays over a fortnight is the leading indicator of alert fatigue or irrelevance, regardless of what the model thinks of its own output.

Follow-up rate. Did a finding spark an investigation — a chat-thread question, a metric drill-down, a teammate Slack? This is the closest proxy for "the system surfaced something useful" that doesn't require explicit feedback.

Not-relevant rate. When we add a thumbs-down or "hide this" affordance, what fraction of findings get dismissed? Per finding type and per customer segment, this is the input signal for replacing the business-weight defaults with learned per-customer weights.

7-day retention after first digest. Does the digest change the morning habit, or is it tried-and-abandoned? This is the single most important number; everything else is a leading indicator of this one.

Alert-to-action conversion. When the system flags something with an "investigate today" question, did the user take an action that's traceable in the data (paused a campaign, changed a budget, opened a checkout audit)? Hard to measure without integration, but it's the metric a CFO would actually care about.

I would deliberately not measure language-model quality scores (BLEU, ROUGE, judge-LLM evaluations) for the daily digest. They correlate poorly with whether the brand owner actually uses the product. Engagement is the only honest metric.